

Functional Programming – 5 credits - course description

The subject last year was announced in the Autumn and the Spring semester as well. It consists of 2h course, 2h practice, and 1h consultation.

Syllabus

The students following this subject will acquire knowledge about fundamentals of functional programming languages. The functional programming notions are studied extensively. In order to get familiar with functional programming language elements the state-of-the-art Haskell pure and lazy functional programming language is used. This is an excellent language to initiate students in all major functional programming areas, as it contains and supports all the essential FP language elements and tools. Therefore, many universities still consider as best practice to teach a functional language as the first programming language.

The students, after finishing successful the subject, are able to solve typical problems and to program them in functional language; however, to become expert in functional programming would need more semesters' studies. The language is used therefore as a tool to study more general knowledge of the functional programming paradigm and related applications.

It is of high importance to understand functional language elements and tools, and to use them consciously when solving programming problems, to use the adequate methods and techniques to achieve high quality coding with well-known functional programming conventions which results in readable and reusable general codes.

Studying functional programming helps in forming a correct programming style with the necessary programming language knowledge base with enhancement of FP programming beauties.

It is regarded compulsory to actively take part in courses and practices, to conduct individual studies helped by consultations too. Please use all the opportunities provided by this subject to improve your programming skills, and to develop your computer science studying techniques. It is of your own responsibility to study every week's material, to do the homeworks, and prepare for upcoming classes.

Course content

- Introduction and motivation, FP history
- defining functions, guards, recursive functions, elementary types
- syntax, reduction steps, redex, normal form, lazy and eager evaluation
- standard functions, types, predefined operators
- data structures in FP - lists, tuple, record, array, class
- lists, definitions, operations and functions on lists, generators, patterns, recursion
- Fibonacci function, double recursion
- types in FP - simple types, algebraic types, composite types, trees

- algorithms - search, sort, traverse
- higher order functions, filters, map, iterate, fold
- tuples, list comprehensions, equivalences
- sorting algorithms, merging
- infinite lists
- records
- algebraic types, trees
- abstract data types (ADT)
- classes, instances, type classes
- FP applications.

Requirements

During the semester theoretical and practical knowledge acquiring are assessed. The course offers 5 credits when successfully completed. This is equivalent to approximately 150 working hours, which can be divided as in the followings: 52 h (2h course + 2h lab during approx. 13 weeks), 65 h individual studies hours (5 hours/week for solving weekly homework including consultations and preparation for practices), 16-17 hours' preparations for mid-term and for end-term programming exams.

Preconditions

- *Presence* at courses (**Wednesday 8.15-9.45 a.m. or Thursday 12.15.-1.45 pm 0.821 Bolyai János**) and mandatory attendance at practices (see the **14 groups** in Neptun) with max 3-3 missed attendances, and optional attendance at consultations **Friday 5-6pm 0.308 Lovarda** and in consultation teams.

Assessment

- Weekly **mini-quizzes** of 5 theoretical questions at the beginning of the lecture. At the end of semester out of $5 \times 8 = 40$ points min. 20 points must be collected (starting on the week of 30th September).
- Weekly **progress tasks** (starting on the week of 30th September) during practice lab sessions, consisting of one programming assignment similar to previous week's practice tasks and suggested homeworks with their upload before deadline. A program is accepted if it fully complies with the specifications. At the end of semester out of 8 progress task class works min. 4 should be good, i.e. accepted.
- Individual implementation of the optional HOMEWORK assignments (8 *homeworks*) and their upload before their deadline.
- **Mid-term NOV 13 or 14 mostly at lecture times (we will announce the division of the lecture team, retakers possibly a Friday before that week - TBA)** including 20 *theoretical quiz* questions (20 x 1 points) and solving *programming problems* (10 x 100 total points) on lab computers.
- **End-term DEC 11 or 12 at lecture times (we will announce the division of the lecture team, retakers possibly a Friday before that week - TBA)** including 10

theoretical quiz questions (10 x 1 points) and solving *programming problems* (10 x 100 total points) on lab computers.

- To obtain a mark, weekly work, attendance and good points at midterm and at endterm is compulsory.
- The **final points** are computed by 40% of mid-term programming, 40% of end-term programming, 10% of the two theoretical exam quizzes (sum of both exam quizzes of midterm and endterm) and 10% of the weekly progress tasks (sum of 8 PTs).
- The **final mark** is based on the final points achieved as in the following:
mark 2 for 50-60% of maximum points; mark 3 for 61-70%; mark 4 for 71-85%; mark 5 for 86-100%. It can be calculated only if at each point categories min 50% was obtained. If at any category less than the min. requirement is acquired, then the final mark is 1 (failed).
- We offer one-time **retake for midterm (time tba)** and one-time **retake for endterm (time tba)** for the mark(s) that is (are) missing or for correcting parts of the assessments for a better mark.
- Weekly activities as **progress tasks and mini quizzes cannot be retaken.**
- At end of semester we offer 2 extra progress tasks and 2 extra mini quizzes to replace the missed ones in case of illness or other justified, unforeseen events.

In all EVALUATION categories (mini-quizzes, progress tasks, mid-term programming, end-term programming, theoretical quizzes) **min. 50%** must be achieved.

Copied class works **are rejected**, **copied or cheated** exam-works are **rejected** and the subject needs to be retaken.

In pseudocode:

$X := \text{Avg}(\text{SumAllProgressTasks}) * 10\% + (\text{MidTermQuiz} + \text{EndtermQuiz}) * 10\% +$
 $\text{MidTermExam} * 40\% + \text{EndTermExam} * 40\%$

case X: X > 85 : FinalMark := **5** passed
85 >= X > 70 : FinalMark := **4** passed
70 >= X > 60 : FinalMark := **3** passed
60 >= X >= 50 : FinalMark := **2** passed
50 > X : FinalMark := **1** failed

if $\text{SumAllProgressTasks} < 50\%$ or $\text{SumAllMiniQuizzes} < 50\%$ or
 $(\text{MidTermQuiz} + \text{EndtermQuiz}) < 50\%$ or $\text{MidTermExam} < 50\%$ or $\text{EndTermExam} < 50\%$
then failed, needs retaking the subject.

References

- Jeremy Gibbons: Fun of programming, Palgrave Macmillian, 2003.
- Chris Okasaki: Purely Functional Data Structures, Cambridge University Press, 1999.
- Richard Bird: Pearls of Functional Algorithm Design, Cambridge University Press, 2010.
- Greg Michaelson: An Introduction to Functional Programming Through Lambda Calculus, Dover Books on Mathematics, 2011.
- and many other FP, Haskell, ML, F# etc. books see the readings.

Readings:

<https://discourse.haskell.org/t/the-best-haskell-related-materials-to-read-in-2024/8739>

<https://www.extrema.is/articles/haskell-books> 77 best books list

<https://code.world> 9 & <https://code.world/haskell> 29 best hands-on learning site

<https://blog.plover.com/prog/haskell/numbers.html> 9 numeric types

<https://wiki.haskell.org/Typeclassopedia> 5 typeclasses

<https://www.fpcomplete.com/haskell/tutorial/all-about-strictness> 5 strictness / laziness

<https://github.com/Gabriella439/post-rfc/blob/main/sotu.md#state-of-the-haskell-ecosystem> 9 ecosystem overview

https://wiki.haskell.org/Haskell_in_industry 2 & <https://haskellcosm.com> 8 who's using haskell

https://downloads.haskell.org/ghc/latest/docs/users_guide 1, <https://cabal.readthedocs.io> 2 & <https://docs.haskellstack.org> 4 secret knowledge for wizards

<https://hackage.haskell.org/package/base/docs/Prelude.html> 1 & <https://hackage.haskell.org/package/base> 1 more secret knowledge for uplevelling

<https://wiki.haskell.org> & <https://en.wikibooks.org/wiki/Haskell> not the same, both informative

<https://www.fpcomplete.com/haskell/syllabus> 2 & <https://www.fpcomplete.com/haskell/learn/> 1 practical guides for intermediate haskelling

<https://haskell.foundation/hs-opt-handbook.github.io> 12 optimising

<https://haskell.foundation/podcast> 1 interviews

<https://haskell.foundation/projects> 2 more Haskell Foundation activities

<https://www.stephendiehl.com/posts/docs.html> 28 Sunsetting “What I Wish I Knew”

<https://github.com/sdiehl/wiwinwlh> 14 source

https://wiki.haskell.org/Research_papers/Functional_pearls

<https://simonmar.github.io/pages/pcph.html>